MERN STACK WEB APPLICATION

FinanceTrack

Personal Finance Management System

S.NO	Name of the student	Register Number
1	K.B.Charan	99230040358
2	V. Shiva Kumar	99230041082
3	A.Sushma	99230040251
4	V.Padma	99230040802
5	B. Harsha	99230040080

Problem Statement

Managing money becomes difficult when financial records are scattered.

Manual tracking

Income and expenses are often recorded in notebooks or separate sheets.

Low spending visibility

Users may not clearly see where their money goes each month.

Budget overruns

Without timely monitoring, category spending can exceed planned limits.

Difficult reporting

Preparing monthly summaries and comparing expenses takes time.

Proposed Solution

FinanceTrack brings everyday personal finance tasks into one website.

- Record income and expenses in one place
- Organize transactions by category and date
- Set monthly budgets for expense categories
- View balances, trends and category summaries
- Export transaction reports as CSV
- Manage user accounts and system categories (Admin)

Track

Record income and expenses

Control

Monitor category budgets

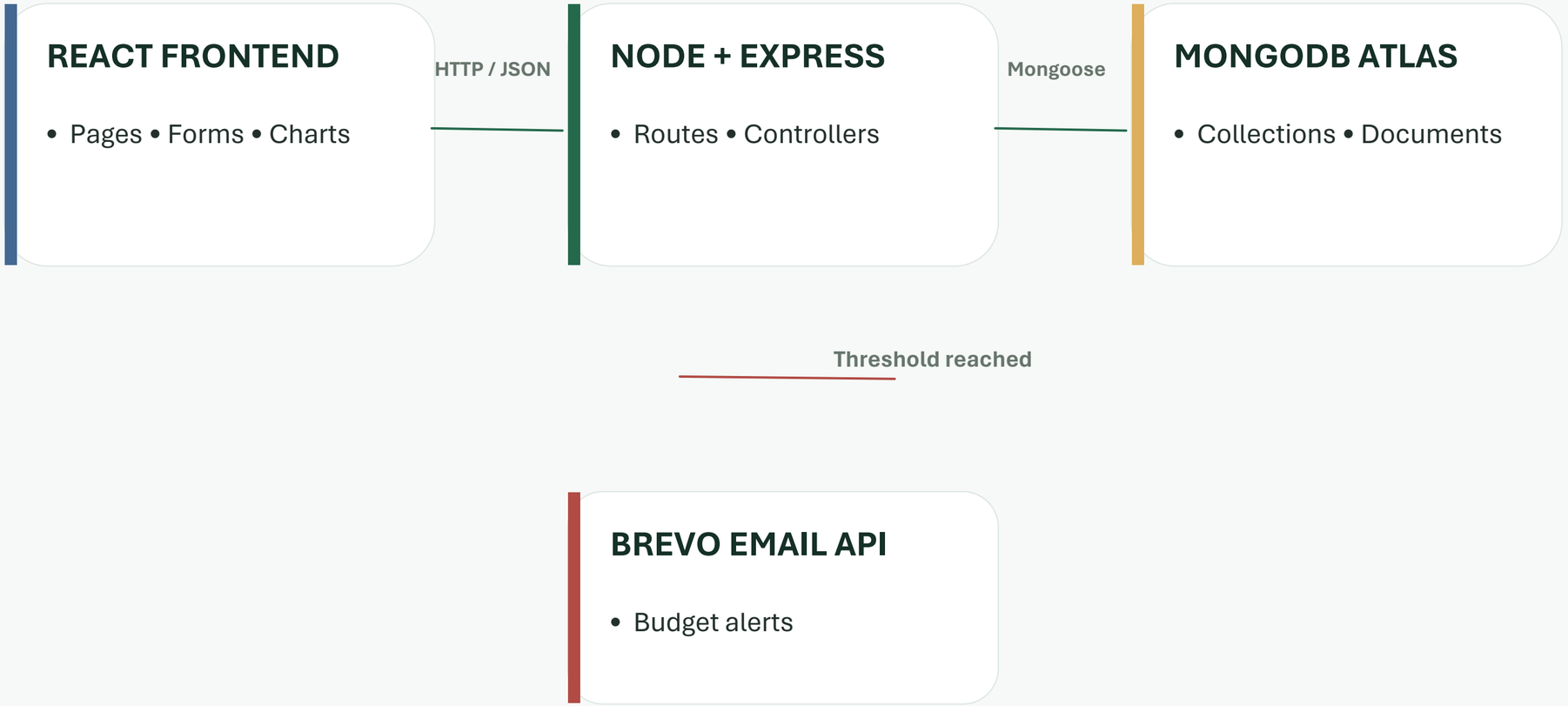
Understand

View charts and reports

Technologies Used

FRONTEND	React 18 • Vite • React Router • Axios • Recharts
BACKEND	Node.js 18 • Express.js • JWT • bcryptjs
DATABASE	MongoDB Atlas • Mongoose
SUPPORT	Brevo API • React Hot Toast • date-fns
DEPLOYMENT	Vercel (frontend) • Render (backend)

System Architecture



Request flow: React → Express API → Mongoose → MongoDB → JSON response to React

How Each MERN Technology Is Used in FinanceTrack

React — Website interface

- Renders Login, Register and dashboard pages
- Builds transaction, budget and profile forms
- Uses React Router for page navigation
- Uses Recharts to display financial summaries

Node.js — Backend execution

- Runs the FinanceTrack server process
- Executes authentication and finance logic
- Runs transaction and budget controllers
- Triggers budget-check service after transaction changes

Express.js — API layer

- Defines /api/auth, /api/transactions and /api/budgets
- Applies authentication middleware to protected routes
- Routes requests to the correct controller
- Returns JSON responses and handles errors

MongoDB — Application data

- Stores users, transactions and categories
- Stores monthly budgets and alert records
- Links records using MongoDB ObjectId references
- Mongoose validates documents and supports queries

Pages Available in the Website

User-facing pages

- Landing page
- Register and Login
- Dashboard
- Transactions
- Budgets and Budget Analysis
- Reports

Account and admin pages

- Profile
- Change Password
- Admin Dashboard
- User Management
- Category Management

Main navigation groups financial activity separately from account and administration tasks.

- Users manage their own financial records and budgets.
- Administrators manage users, system categories and application statistics.

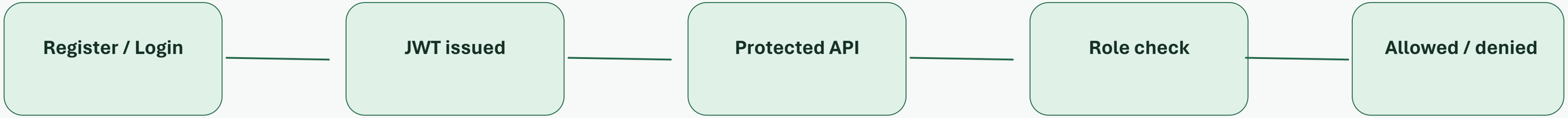
Authentication & Authorization

Authentication — Who are you?

- Register with name, email and password
- Password is hashed using bcryptjs
- Login verifies password and issues a JWT
- Axios sends token with protected API requests

Authorization — What can you access?

- USER and ADMIN roles
- protect middleware verifies the JWT
- requireAdmin restricts admin-only APIs
- User records are scoped to the signed-in account



JWT = signed access token • 401 = unauthenticated • 403 = authenticated but not authorized

Functional Requirements

User requirements

- Register, log in and manage profile
- Create, view, edit and delete transactions
- Create and manage monthly category budgets
- Filter transaction records and view reports

System / admin requirements

- Authenticate protected API requests
- Validate user input and handle errors
- Calculate dashboard and budget summaries
- Send budget threshold email notifications
- Allow admins to manage users and categories

Non-functional expectations

- Security: hashed passwords, JWT and role-based access
- Usability: responsive pages, clear feedback and visual summaries
- Maintainability: modular routes, controllers, models and reusable components

Key Features

Dashboard

Income, expenses, balance and visual summaries

Transactions

Add, edit, delete, search and filter records

Budgets

Monthly category limits and usage analysis

Reports

Date/category filters and CSV export

Categories

System categories and personal categories

Admin panel

User management, categories and statistics

Automated Budget Alert Workflow

1. Expense added / updated / deleted
2. Backend recalculates category spending
3. Compare spending with monthly budget
4. At 90% → warning email via Brevo
5. At 100%+ → limit reached email
6. Save alert status in MongoDB

Alert records help avoid sending the same threshold notification repeatedly.

One place to track, plan and understand money.

- Combines income, expense and budget management
- Connects React frontend with Express REST APIs
- Stores finance data in MongoDB Atlas
- Adds automatic budget alerts and visual reports

Live website: https://frontend-tau-livid-99.vercel.app/

Thank You